Preston-Check Moat-Building Strategy

Building defensible IP, AI leverage, and best-in-class UX

Preston-Check Maintainers

2026-05-04

Moat-Building Strategy

What is actually defensible in an open-source security tool?

The catalog itself is not a moat. Anyone can copy 294 bash scripts. The framework citations are not a moat — they come from public regulations. The brand and trademark provide some defense but are weak alone. The real moats compound over time and have to be built deliberately, starting now, to have anything in 18 months.

Five categories of moat in order of leverage:

Aggregated anonymized telemetry data is the highest-leverage asset because it is the one thing competitors literally cannot copy. As opt-in scans accumulate, Preston-Check builds a dataset nobody else has: which check categories fail most often across fintechs, how scores trend over time, what differentiates top-quartile codebases from bottom-quartile, regional patterns, and emerging vulnerability frequency. After year one this becomes the canonical “State of Fintech Security” benchmark. After year three, customers buy the dashboard primarily to see how they compare to their peer cohort. Snyk’s data moat — eight years of curated vulnerability database — is what makes their valuation defensible despite open-source competitors.

Proprietary fintech-specific threat intelligence layer, daily-updated. Read every CVE published, every fintech post-mortem, every regulatory enforcement action, every published incident report (Bybit Feb 2025, GMX July 2025, Cetus integer overflow, Wormhole bridge). For each, distill into one or more new check patterns. After eighteen months your catalog is not just “OWASP plus frameworks” but

"OWASP plus frameworks plus 200+ checks distilled from real-world fintech breaches in 2026-2027." That is the moat — and the work done to maintain it justifies the Enterprise pricing because customers know they get the curated stream.

Auditor relationships and SaaS integrations. Once Big Four and mid-tier auditors accept Preston-Check evidence packets in their workflow, switching costs compound. Same for compliance platform integrations (Drata, Vanta, Secureframe). Each integration takes 2-4 weeks to build and 6 months to land contractually, but each one adds permanent friction for any competitor. Snyk has approximately 40 such integrations; small competitors have 0-3.

An AI layer that improves with scan volume. The model learns from human-confirmed findings versus false positives and gets sharper over time. Competitors copying your checks do not get the model; they have to retrain from scratch with much less data. This is the deepest technical moat available in 2026.

Reputation and canonical-source positioning. Preston-Check's name in conversations like "do you have a Preston-Check report?" The OWASP Foundation, FATF, and regulators eventually citing it as the reference implementation. This takes years and lots of conference talks but is genuinely defensible once established.

AI / ML models worth building

Specific list, ordered by ROI. The first three are cheap and ship fast; the last three are the hard moats.

Tier 1: Ship in the first 6 weeks

False-positive filter (✅ shipped v1.6.0, wired in v1.7.1). Every finding goes through a small LLM that takes the file context plus the finding and judges "real vulnerability" / "likely false positive" / "needs human review." Cuts alert fatigue by 30-50% based on what Snyk Code, Semgrep Pro, and CodeQL deliver. Lowest cost (a single API call to Claude or GPT per finding, or a fine-tuned 7B model running locally), highest immediate UX impact. Customers feel this on first scan. *Shipped as* `lib/ai_analyze.sh`*; activated via* `--ai-augment` *after the v1.7.1 runner wiring.*

Auto-fix generator (✅ shipped v1.7.1). Given a finding plus surrounding code, generate a patch that resolves the issue. Critical UX moment: not just "you have SQL injection at line 42" but "click here to apply the fix" with a diff preview. **Auto-fix is the number one feature converting Snyk free users to paid.** *Shipped as* `lib/ai_autofix.sh`*; activated via* `--ai-fix` *(which implies* `--ai-augment`*). Capped at 5 findings per check to bound scan time, per-finding cached, gracefully no-ops without API keys.*

Custom check synthesizer (~3 weeks, not yet started). Natural language to working `.sh` check file. "Write me a check that detects when a transaction's idempotency key uses Math.random() instead of crypto.randomUUID()." The LLM generates the check, runs it against a fixture corpus, iterates until it passes the lint gate, then opens a PR to `checks/community/proposed/`. Lowers the barrier for community contributions enormously and is itself a viral lever.

Tier 2: Six-month roadmap

AST-aware semantic analysis (~2 months). Tree-sitter parses code into syntax trees; a fine-tuned small model reasons over the AST. Catches what grep cannot: "this function reads a value from req.body and uses it in a Mongo query without sanitization across three function calls." This is the long-tail accuracy gain that separates Semgrep Pro from Semgrep OSS.

Risk-scoring model with peer comparison (~6 weeks, requires telemetry data). Combines findings plus codebase metadata (industry, stack, exposure) and outputs a calibrated risk score. Compares to anonymized cohort: "your fintech is in the bottom-quartile for sanctions screening among similar-size US payment processors." This becomes the headline number on the dashboard and the executive-summary stat in CISO reports. **It only works if you have telemetry volume**, which is why the data moat is foundational.

Threat-intel auto-synthesis (✅ pipeline shipped v1.6.0, automation wired v1.7.1). Daily ingest of CVE feeds, regulatory enforcement actions, fintech post-mortems, breach disclosures. LLM extracts the attack pattern, generates a candidate check, runs it against the test corpus, surfaces it for maintainer review. After 6 months of operation, the catalog updates itself faster than humans can write checks. *Foundation shipped as `tools/sync-threat-intel.py`. Cron + auto-PR shipped as `.github/workflows/threat-intel-sync.yml`. Verified end-to-end against live NIST NVD: 339 CVEs in a 2-day window, 135 fintech-relevant, drafts emit cleanly into `checks/community/proposed/`. The next leverage point is the Admin Portal's triage UI — see `docs/portals-and-kpis.md`.*

UX: Instant Gratification, Then the Rabbit Hole

The right product UX for a fintech security tool is two-act. Act one is instant: in the first 60 seconds, a developer should know whether their codebase is okay. Act two is the rabbit hole: when they want to dig in, every finding becomes a gateway to deeper context, related findings, framework controls, real-world incidents, and applicable fixes.

Instant gratification (first 60 seconds)

A single shell command from any directory produces a colored terminal report, a numerical security score, and the top three actionable findings with file, line, and fix. We have most of this; the missing pieces are the **score** ("you're at 87/100"), the **grade** ("A-, top quartile of fintechs your size"), and a

single-key action to dive deeper (`preston-check report --open` launches the local browser-based dashboard). The badge for the README, the GitHub Action PR comment, the headline number on the CLI — these are the moments that make people say "oh this is good" within seconds.

Score-as-a-headline is the most underrated UX pattern in security tooling. SonarQube's "A through E" grade is what made it spread inside enterprises in the 2010s — every developer instinctively knew "B is fine, D is bad" without reading documentation. Preston-Check has the data to compute a calibrated score. Adding a single `Grade: A-` line at the top of every report is a one-day change with outsized impact on adoption and shareability.

The rabbit hole (the next 60 minutes)

The local browser-based dashboard at `preston-check web` opens a Next.js app served from a localhost worker that reads the latest report.json. Heatmap of where issues cluster in your codebase. Click any finding and a modal shows the source code with the offending line highlighted, the framework controls it touches, related findings nearby, real-world incidents that exploited this pattern, and a one-click "apply suggested fix" that generates a patch via the AI auto-fix model. Time-travel slider: "your score 30 days ago vs today." Trend graph per check category. Filter by framework, severity, file directory.

Story mode is the high-engagement pattern most security tools miss. Each finding links to a 2-3 paragraph narrative: "P-308 Bridge Replay Protection — in February 2022, the Wormhole bridge lost $320M because a similar pattern in their cross-chain message handler accepted unsigned messages. Here's what their code looked like; here's how yours compares." This converts findings from chores into education. Developers share these on Twitter; CISOs reference them in security-awareness training; auditors quote them.

Comparison-to-peers is the rabbit hole's deepest layer and the data-moat payoff. "Your fintech scores in the 67th percentile for cryptography hygiene among US payment processors of your scale; 91st percentile for incident response; 23rd percentile for supply-chain security." These percentiles only exist because you have aggregated telemetry. Customers will subscribe just to see them.

What is next, in priority order

Weeks 1-3 — finish the launch. Public repos ✅, landing page (in progress, see `web/landing/`), production keypair, GitHub Marketplace listing, soft launch to fintech communities, coordinated Hacker News and Product Hunt launch. This is the *adoption* phase — get to 1000 free users.

Months 2-3 — AI false-positive filter and auto-fix generator. ✅ Both shipped in v1.7.1. The filter wires through `--ai-augment`; the auto-fix wires through `--ai-fix`. Both customers tell each other about ("the AI explanations are weirdly good", "it actually wrote the patch for me") — that is the viral compounder.

Months 3-5 — SaaS dashboard v1. The browser-based "rabbit hole" experience. Instant gratification (score, grade, top issues) front-and-center; rabbit-hole drill-downs progressively disclosed. Stripe billing. Multi-repo aggregation. Branded PDF export. **First paying customers go through this.** *Designed in `docs/portals-and-kpis.md`; build sequence Q3 2026.*

Months 5-8 — proprietary intelligence layer. Telemetry pipeline live (Worker exists at `workers/telemetry/`; needs `wrangler deploy`). Threat-intel auto-synthesis generating new check candidates weekly ✅ (`.github/workflows/threat-intel-sync.yml`). Peer-comparison percentiles in dashboard. First annual State of Fintech Security report drafted from accumulating data — methodology and template shipped as `docs/state-of-fintech-security/2026.md`.

Months 8-12 — integrations and auditor relationships. Drata, Vanta, Secureframe push Preston-Check evidence into their compliance workflows. Big Four partnership conversations open. Custom-check authoring service rolls out for Enterprise tier. Adapter scaffolds shipped at `tools/integrations/{drata,vanta,secureframe}/`.

The strategic logic: **adoption first, AI second, dashboard third, data moat fourth, integrations fifth.** Reversing that order — building the dashboard before users exist, or chasing integrations before AI quality is good — is how most open-core companies fail. Snyk did it in roughly this order. So did Vercel, Supabase, GitLab.

The single highest-leverage move you can make today

Deploy the telemetry endpoint to Cloudflare and emit the first opt-in scan. The plumbing is shipped (Worker, schema, KV+D1) and the documentation is done (`docs/telemetry.md`); the only missing step is `wrangler deploy` against the Cloudflare account, plus DNS for `telemetry.preston-check.com`. Day one of telemetry collection is day one of the data moat compounding.

The competitive position becomes inarguable around month 12 once you have AI-curated checks fed by anonymized industry data. At that point Preston-Check is no longer "another scanner with regex patterns" but the canonical fintech-security intelligence layer — with the open scanner as the funnel and the SaaS as the monetization.